

OOP Prensipleri

Kod okunabilirliğini ve kullanımını kolaylaştıran prensiplerdir.

OOP Temel Sözlüğü

Class: Objeleri oluşturmak için yazılan template yapısıdır. Objectte sahip olunması istenilen özellik ve metodlar buraya yazılır.

Obje: Class yapısı içinde yazılan özellik ve metodların kullanıma dönüştüğü yapıdır.

Constructorlar objeler oluşturulduğunda initilize eder. Bazı constructor type'ları:

Default Constructor, Parameterized Constructor, Copy Constructor, Move Constructor...

! Private Constructor: Dışarıdan yeni obje oluşumunu engelleyen gizli erişimli değiştiricilere (modifier) sahip olan constructordur.

A constructor declared with the private access modifier that prevents object creation from outside the class

1- Abstraction

Kodun Abstraction bir tüm kompleksite yönetim türüdür. Yapısından ötürü çok kompleks olan bir şeyi dönüştürüp çalışması çok kolay bir hale getirebilir.

Kodun çıktısında önemli olan kodda ne yazıldığı değil kodun ne yaptığıdır. Abstraction'da koddaki detaylar gizlenir, fonksiyonuna odaklanır. Örneğin bir fonksiyon kullanmak istediğimizde fonksiyonu çağırmamız bir tür abstractiondur, her fonksiyonu kullandığımızda en başından fonksiyonu define etmektense parametre(if any) ve fonksiyon ismi ile fonksiyonu çağırıp işlevselliğini kullanırız. Bu sayede kodun kompleksitesini azaltmış ve kullanımı kolaylaştırmış oluruz.

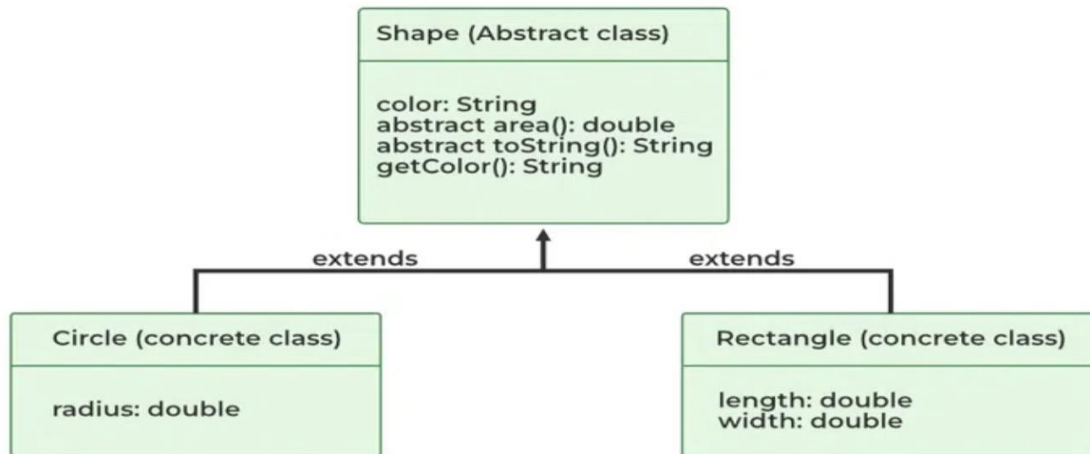
- *It helps users focus on what an object does rather than how it does it.*

How to Achieve Abstraction?

- 1- Abstract Classes (Partial Abstraction)
- 2- Interface

1.1 Abstract Classes:

Direkt objeye sahip olamaz ve abstract fonksiyonlara (gövdesi olmayan fonksiyon) sahip olabilir.



```
abstract class Shape{
    String color;

    // these are abstract methods
    abstract double area();
    public abstract String toString();

    // abstract class can have the constructor
    public Shape(String color){

        System.out.println("Shape constructor called");
        this.color = color;
    }

    // this is a concrete method
    public String getColor(){

        return color;
    }
}
class Circle extends Shape{

    double radius;

    public Circle(String color, double radius){

        // calling Shape constructor
        super(color);
        System.out.println("Circle constructor called");
        this.radius = radius;
    }

    @Override double area(){

        return Math.PI * Math.pow(radius, 2);
    }
}
```

```

@Override public String toString(){

    return "Circle color is " + super.getColor()
        + "and area is : " + area();
}
}
class Rectangle extends Shape{

    double length;
    double width;

    public Rectangle(String color, double length,
        double width)
    {
        // calling Shape constructor
        super(color);
        System.out.println("Rectangle constructor called");
        this.length = length;
        this.width = width;
    }

    @Override double area() { return length * width; }

    @Override public String toString()
    {
        return "Rectangle color is " + super.getColor()
            + "and area is : " + area();
    }
}
public class Test {
    public static void main(String[] args)
    {
        Shape s1 = new Circle("Red", 2.2);
        Shape s2 = new Rectangle("Yellow", 2, 4);

        System.out.println(s1.toString());
        System.out.println(s2.toString());
    }
}

```

1.2 Interface

Interface bir class yapısı için tüm gerekli olan fonksiyonların listelendiği yapı tipidir. Java dilince çok kullanılır.

```
interface Shape{
    // Abstract method for calculating the area
    double calculateArea();
}

// Implement the interface
// in a class named Circle
class Circle implements Shape{

    private double r;

    // Constructor for Circle
    public Circle(double r){

        this.r = r;
    }

    // Implementing the abstract method
    // from the Shape interface
    public double calculateArea()
    {
        return Math.PI * r * r;
    }
}

// Implement the interface in a
// class named Rectangle
class Rectangle implements Shape{

    private double length;
    private double width;

    // Constructor for Rectangle
    public Rectangle(double length, double width){

        this.length = length;
        this.width = width;
    }

    // Implementing the abstract
    // method from the Shape interface
    public double calculateArea() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {

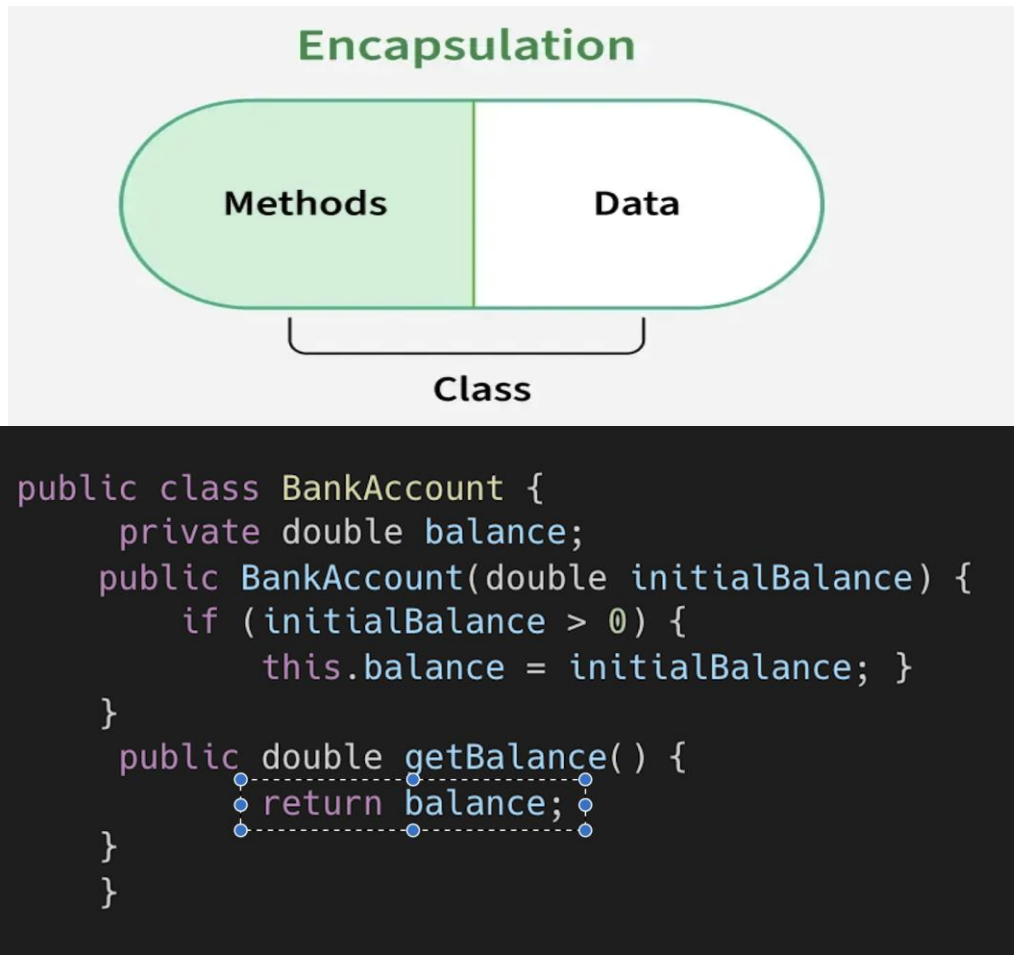
        // Reference type is the interface (Shape)
        Shape cir = new Circle(5.0);
        Shape rect = new Rectangle(4.0, 6.0);

        // Dynamic method dispatch – decides which method to call at runtime
        System.out.println("Area of Circle: " + cir.calculateArea());
        System.out.println("Area of Rectangle: " + rect.calculateArea());
    }
}
```

2- Encapsulation

(Encapsulation) Kapsülleme, bir nesnenin durumunu temsil eden verilerin (niteliklerin/değişkenlerin) dış dünyadan **gizlenmesi** ve bu verilere erişimin yalnızca sınıf tarafından belirlenen güvenli metotlar yani erişimciler ve değiştiriciler (*getter/setter*) aracılığıyla sağlanması prensibidir.

- **Amacı:** Veri bütünlüğünü (data integrity) sağlamak, nesnenin iç durumunun yetkisiz veya hatalı dış müdahalelerle bozulmasını engellemektir.
- **Uygulama:** Sınıf içindeki veriler private (özel) erişim belirleyicisi ile tanımlanırken, bu verilere kontrollü erişim sağlayan metotlar public (genel) olarak tanımlanır. Bu sayede nesnenin iç işleyişi ile dış arayüzü birbirinden yalıtılır.



3- Polymorphism

Çok biçimlilik (Polymorphism), nesne yönelimli programlamada (OOP) farklı sınıflara ait nesnelerin, ortak bir üst sınıf (base class) veya arayüz (interface) referansı üzerinden çağrılan aynı metoda, kendi iç yapılarına özgü farklı tepkiler verebilmesi prensibidir. Literatürde "tek arayüz, çoklu yöntem" (one interface, multiple methods) yaklaşımı olarak da adlandırılır.

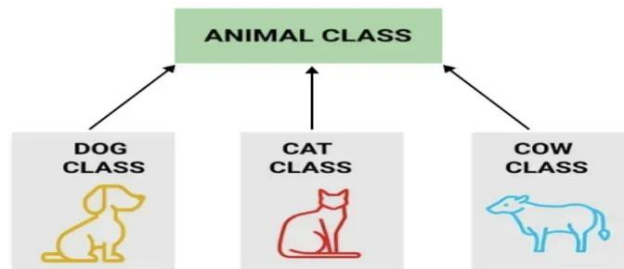
```
public abstract class Shape {  
  
    public abstract double calculateArea();  
}  
  
public class Circle extends Shape {  
  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
}
```

```
@Override  
public double calculateArea() {  
    return Math.PI * radius * radius;  
}  
  
public class Square extends Shape {  
  
    private double side;  
  
    public Square(double side) {  
        this.side = side;  
    }  
  
    @Override  
    public double calculateArea() {  
        return side * side;  
    }  
}
```

4- Inheritance

Kalıtım (Inheritance), nesne yönelimli programlamada (OOP) yeni oluşturulan bir sınıfın (alt sınıf / *child class*), halihazırda var olan başka bir sınıfın (üst sınıf / *parent class*) niteliklerini (değişkenler) ve davranışlarını (metotlar) doğrudan devralması prensibidir. Sınıflar arasında hiyerarşik ve mantıksal bir "IS-A" (**bir türüdür**) ilişkisi kurar.

Temel Amacı ve İşlevi: Kalıtımın temel amacı **kodun yeniden kullanılabilirliğini (code reusability)** artırmaktır. Ortak özellikler ve fonksiyonlar tek bir temel sınıfta (base class) tanımlanır; türetilen alt sınıflar bu özellikleri tekrar yazmaya gerek kalmadan miras alır. Alt sınıflar yalnızca kendilerine özgü (spesifik) yeni özellikleri veya değiştirilmiş davranışları barındırır. Bu mimari, yazılımın bakımını (maintenance) kolaylaştırır ve kod tekrarını (redundancy) büyük oranda ortadan kaldırır.



```
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle" << endl;
    }
};

class Car : public Vehicle {
public:
    Car() {
        cout << "This Vehicle is Car" << endl;
    }
};

int main() {
    Car obj;
    return 0;
}
```

Output//

"This is a Vehicle"

"This Vehicle is Car"